

Strings

When characters like letters, digits, and symbols are arranged in a sequence, it is known as a string. The usual string object is an immutable (unchangeable) sequence of characters accessed by offset (position).

Strings are immutable, but you can update the value of a string by assigning a new string to the same string variable.

```
In [1]: s="welocme LJU"
print(id(s))
print(s)
#you can update the value of a string by assigning a new string to the same string
s=s+"2022"
print(id(s))
print(s)
```

```
2368209510256
welocme LJU
2368209522480
welocme LJU2022
```

We will cover the following topics in this chapter:

Creating a string

Accessing values in string

Changing or deleting a string

String operations

Build-in string methods

strings are a sequence of Unicode Characters

Unicode string

All text is unicode text, including the text encoded with one character per byte (8 bits) in the ASCII scheme. Python supports richer character sets and encoding schemes with unicode, that is, strings that may use multiple bytes to represent characters in memory, and which translate text to and from various encodings on files.

Creating a string

We can create strings by enclosing characters within a single quote or double quotes. Python supports triple quotes, but these are generally used to represent multiline strings and docstrings.

In Python strings, single- and double-quote characters are interchangeable, i.e., string literals can be expressed in two single or two double quotes. Here, the two representations work in the same way and return the same type of object.

The reason for supporting the two representations is that it allows programmers to enclose a quote character of the other type within a string without a backslash. It also helps programmers to enclose a single-quote character in a string enclosed in double-quote characters, and vice-versa.

```
In [2]: s="welcome"
        print(s)
```

welcome

```
In [4]: s='welcome'
        print(s)
```

welcome

```
In [5]: #for multiline string
        s='''welcome'''
        print(s)
```

welcome

```
In [6]: s="""hello
        welcome"""
        print(s)
```

hello
welcome

```
In [7]: s='it's blowing outside'
        print(s)
```

```
File "C:\Users\VISHAL\AppData\Local\Temp\ipykernel_13484\1923883316.py", line 1
    s='it's blowing outside'
      ^
SyntaxError: invalid syntax
```

```
In [9]: s="it's blowing outside"
        print(s)
```

it's blowing outside

```
In [11]: a=10
          print(a,"--",id(a),"--",type(a))
          a=str(a)
          print(a,"--",id(a),"--",type(a))
```

```
10 -- 2242939873872 -- <class 'int'>
10 -- 2243025646768 -- <class 'str'>
```

Accessing substring from a string

Python does not allow a character type; these are considered as strings of length one, so they are also considered substrings.

We can access individual characters using indexing and with different characters using the process of slicing. The index always starts from zero, but an `IndexError` may arise while trying to access a character out of the index range. The index is always an integer; we cannot use float or other data types; this will lead to `TypeError`.

Python allows negative indexing for its sequences. For example, the index of `-1` indicates the last item, `-2` the second last item, and so on. Python also allows you to access a range of items in a string using the slicing operator (colon).

Slice expressions support the optional third index, which is used as a step (sometimes called stride). Steps are added to the index of each item extracted. Slices are now `X[I:J:K]`, which means that all items in `X` are removed from offsets `I` through `J*1`, by `K`. `K` is generally set to `+1`, which is why all items in a slice are generally extracted from left to right. However, the third limit can be used to skip items or reverse their order if you specify an explicit value. `X[1:10:2]` will fetch every other item in `X` from offsets `1–9`. It will collect items at offsets `1, 3, 5, 7, and 9`. Since the first and second limits are set to `0` and the length of the sequence, respectively, `X[:2]` receives all items from the beginning to the end of the sequence.

In [2]: `# positive Indexing (left to right) (0 to n-1 here n is number of chracter in st`
`s="hello world"`
`print(s[0])`
`print(s[1])`
`print(s[40])`

h
e

```
-----
IndexError                                Traceback (most recent call last)
Cell In[2], line 5
      3 print(s[0])
      4 print(s[1])
----> 5 print(s[40])

IndexError: string index out of range
```

In [13]: `# negative Indexing (right to left) (start from -1 to -n)`
`s="hello world"`
`print(s[-1])`
`print(s[-2])`
`print(s[-40])`

```
d
l
-----
IndexError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_13484\606380217.py in <module>
      3 print(s[-1])
      4 print(s[-2])
----> 5 print(s[-40])

IndexError: string index out of range
```

In [3]: `#slicing`
`s="hello world"`

```
print("1-",s[0:6])
print("2-",s[2:3])
print("3-",s[2:])
print("4-",s[:-1])
print("5-",s[:])
print("6-",s[0:7])
print("7-",s[2:-1])
print("8-",s[:-2])
print("9-",s[1:-1])
print("10-",s[-1:-5])
```

```
1- hello
2- l
3- llo world
4- hello worl
5- hello world
6- hello w
7- llo worl
8- hello wor
9- ello worl
10-
```

```
In [22]: #slicing with step
s="hello world"
print("1-",s[0:6:2])
print("2-",s[2:4:2])
print("3-",s[2::-1])
print("4-",s[:-1:3])
print("5-",s[::2])
print("6-",s[0:7:3])
print("7-",s[2:-1:2])
print("8-",s[:-2:2])
print("9-",s[1:-1:3])
print("10-",s[-1:-5:-2])#negative index starting number grater than ending
print("11-",s[-5::])
```

```
1- hlo
2- l
3- leh
4- hlwl
5- hlowrd
6- hlw
7- lowr
8- hlowr
9- eoo
10- dr
11- world
```

```
In [14]: #slicing with reverse
s="hello world"
print(s[::-1])
print(s[::-2])
```

```
dlrow olleh
drwolh
```

```
In [16]: # Python string slicing: s[start : stop : step]
# start = index where slicing begins
# stop = index where slicing ends, but stop index is NOT included
# step = direction and jump size
#
```

```

# Positive step:
# Moves LEFT to RIGHT
# Example: s[1:8:1]
#
# Negative step:
# Moves RIGHT to LEFT
# Example: s[8:1:-1]

s = "hello world"

print(s[1:-3])
# s[1:-3]
# start = 1 -> 'e'
# stop = -3 -> index of 'r', but stop is not included
# Output: "ello wo"

print(s[-9:12])
# s[-9:12]
# -9 means index 2 -> 'l'
# 12 is beyond the string length, so Python stops at the end safely
# Output: "llo world"

print(s[1:-10], "starting ending same")
# s[1:-10]
# start = 1 -> 'e'
# stop = -10 -> also index 1
# Since start and stop point to the same place, output is empty

print(s[-5:-10], "direction l to r so no output")
# Default step is +1
# Positive step means Python moves left to right
# -5 is index 6 -> 'w'
# -10 is index 1 -> 'e'
# Python cannot move from index 6 to index 1 using +1 direction
# Output is empty

print(s[10:1], "direction r to l so no output")
# Default step is +1
# Positive step moves left to right
# start = 10 -> 'd'
# stop = 1 -> 'e'
# Python cannot move from index 10 to index 1 using +1
# Output is empty

# ----- ADVANCED VERSION -----

print(s[10:1:-1])
# step = -1 means move right to left
# start = 10 -> 'd'
# stop = 1 -> stop before index 1
# Output: "dlrow oll"

print(s[::-1])
# No start, no stop, step = -1
# Reverses the whole string
# Output: "dlrow olleh"

print(s[::2])
# step = 2 means take every 2nd character from left to right

```

```
# Output: "hlowrd"

print(s[::-2])
# step = -2 means take every 2nd character from right to left
# Output: "drwolh"

print(s[-1:-12:-1])
# start = -1 -> last character 'd'
# stop = -12 -> before first character
# step = -1 -> move right to left
# Output: "dlrow olleh"
```

```
ello wo
llo world
starting ending same
direction l to r so no output
direction r to l so no output
dlrow oll
dlrow olleh
hlowrd
drwolh
dlrow olleh
```

Editing and deleting in string

Strings are immutable, which implies that elements of a string once assigned cannot be changed. Python allows you to set the same name to different strings.

```
In [17]: s="hello world"
s[0]="H"
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[17], line 2
      1 s="hello world"
----> 2 s[0]="H"

TypeError: 'str' object does not support item assignment
```

```
In [18]: s="hello world"
print(id(s))
del s
print (id(s))
```

```
2368281094192
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[18], line 4
      2 print(id(s))
      3 del s
----> 4 print (id(s))

NameError: name 's' is not defined
```

```
In [19]: s="hello world"
del s[:]
print(s)
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[19], line 2
      1 s="hello world"
----> 2 del s[:]
      3 print(s)

TypeError: 'str' object does not support item deletion

```

Operations on Strings

Arithmetic Operations

Relational Operations

Logical Operations

Loops on Strings

Membership Operations

```
In [20]: print("gandhinagar"+" "+"delhi")
```

gandhinagar delhi

```
In [21]: print(""*50)
```

```
In [22]: str="hello"
print(id(str))
str+="student"
print(str)
print(id(str))
```

2368188834160

hellostudent

2368278795312

Relational Operations

```
In [23]: "delhi"=="gandhinagar"
```

Out[23]: False

```
In [24]: "delhi"=="delhi"
```

Out[24]: True

```
In [25]: 'delhi' != 'delhi'
```

Out[25]: False

```
In [26]: 'mumbai' > 'pune'
# Lexiographically
```

Out[26]: False

```
In [27]: 'Pune' > 'pune'
```

```
Out[27]: False
```

Logical Operations

```
In [28]: 'hello' and 'world'
```

```
Out[28]: 'world'
```

```
In [29]: 'hello' or 'world'
```

```
Out[29]: 'hello'
```

```
In [30]: '' and 'world'
```

```
Out[30]: ''
```

```
In [31]: '' or 'world'
```

```
Out[31]: 'world'
```

```
In [32]: not 'hello'
```

```
Out[32]: False
```

```
In [33]: not ""
```

```
Out[33]: True
```

Loops on Strings

```
In [34]: str="hello "  
index=0  
for i in str:  
    print('str [' ,index,"] = ",i)  
    index+=1
```

```
str [ 0 ] = h  
str [ 1 ] = e  
str [ 2 ] = l  
str [ 3 ] = l  
str [ 4 ] = o  
str [ 5 ] =
```

```
In [49]: str="hello "  
for i in str:  
    print("python")
```

```
python  
python  
python  
python  
python  
python
```



```
In [35]: str="hello "  
for index,j in enumerate(str):  
    print(index,"--",j)
```

```
0 -- h  
1 -- e  
2 -- l  
3 -- l  
4 -- o  
5 --
```

```
In [36]: str="hello "  
for i,j in enumerate(str):  
    print(i,"--",j)
```

```
0 -- h  
1 -- e  
2 -- l  
3 -- l  
4 -- o  
5 --
```

Membership Operations

```
In [37]: 'D' in 'delhi'
```

```
Out[37]: False
```

```
In [38]: "l" in "mango"
```

```
Out[38]: False
```

```
In [39]: "l" not in "mango"
```

```
Out[39]: True
```

Common Functions

len

max

min

sorted

String len() method

This len() method returns the length of the string.

reversed()

Syntax:

len(str)

Parameters

NA

return

integer value

String max() method

The max() method returns the maximized alphabetical character from the string

Syntax:

max(str)

Parameters

str - From this string, the maximum alphabetical character should be returned

return

give maximized alphabetical character

String min() method

The min() method returns the minimum alphabetical character in the string str.

Syntax:

min(str)

Parameters

str - The string from which the minimum alphabetical character needs to be returned.

Return type

This method returns the minimum alphabetical character from the str string.

sorted()

The sorted() function returns a sorted list of the specified iterable object.

You can specify ascending or descending order. Strings are sorted alphabetically, and numbers are sorted numerically.

Note: You cannot sort a list that contains BOTH string values AND numeric values.

```
In [40]: str="hello word"
          print(len(str))#give total character of string
```

```
print(max(str))#give biggest character according to Ascii value  
print(min(str))#give smallest character according to Ascii value here space has  
str="helloworld"  
print(min(str))
```

l0

w

d

```
In [64]: str="hello word"  
print(sorted(str))
```

[' ', 'd', 'e', 'h', 'l', 'l', 'o', 'o', 'r', 'w']

```
In [65]: str="hello word"  
print(sorted(str,reverse=True))
```

['w', 'r', 'o', 'o', 'l', 'l', 'h', 'e', 'd', ' ']

```
In [41]: str="hello word"  
print(list(reversed(str)))
```

['d', 'r', 'o', 'w', ' ', 'o', 'l', 'l', 'e', 'h']

common function

Function	Usage	Example
<code>count(str, beg, end)</code>	Counts number of times str occurs in a string. You can specify beg as 0 and end as the length of the message to search the entire string or use any other value to just search a part of the string.	<pre>str = "he" message = "helloworldhelloworld" print(message.count(str,0, len(message)))</pre> OUTPUT 3
<code>endswith(suffix, beg, end)</code>	Checks if string ends with suffix; returns True if so and False otherwise. You can either set beg = 0 and end equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>message = "She is my best friend" print(message.endswith("end", 0, len(message)))</pre> OUTPUT True
<code>startswith(prefix, beg, end)</code>	Checks if string starts with prefix; if so, it returns True and False otherwise. You can either set beg = 0 and end equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>str = "The world is beautiful" print(str.startswith("Th",0, len(str)))</pre> OUTPUT True
<code>find(str, beg, end)</code>	Checks if str is present in string. If found it returns the position at which str occurs in string, otherwise returns -1. You can either set beg = 0 and end equal to the length of the message to search entire string or use any other value to search a part of it.	<pre>message = "She is my best friend" print(message.find("my",0, len(message)))</pre> OUTPUT 7
<code>index(str, beg, end)</code>	Same as find but raises an exception if str is not found.	<pre>message = "She is my best friend" print(message.index("mine", 0, len(message)))</pre> OUTPUT ValueError: substring not found
<code>rfind(str, beg, end)</code>	Same as find but starts searching from the end.	<pre>str = "Is this your bag?" print(str.rfind("is", 0, len(str)))</pre> OUTPUT 5
<code>rindex(str, beg, end)</code>	Same as rindex but start searching from the end and raises an exception if str is not found.	<pre>str = "Is this your bag?" print(str.rindex("you", 0, len(str)))</pre> OUTPUT 8

Function	Usage	Example
<code>isalnum()</code>	Returns True if string has at least 1 character and every character is either a number or an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalnum())</pre> OUTPUT True
<code>isalpha()</code>	Returns True if string has at least 1 character and every character is an alphabet and False otherwise.	<pre>message = "JamesBond007" print(message.isalpha())</pre> OUTPUT False
<code>isdigit()</code>	Returns True if string contains only digits and False otherwise.	<pre>message = "007" print(message.isdigit())</pre> OUTPUT True
<code>islower()</code>	Returns True if string has at least 1 character and every character is a lowercase alphabet and False otherwise.	<pre>message = "Hello" print(message.islower())</pre> OUTPUT False
<code>isspace()</code>	Returns True if string contains only whitespace characters and False otherwise.	<pre>message = " "</pre> <pre>print(message.isspace())</pre> OUTPUT True
<code>isupper()</code>	Returns True if string has at least 1 character and every character is an upper case alphabet and False otherwise.	<pre>message = "HELLO" print(message.isupper())</pre> OUTPUT True
<code>len(string)</code>	Returns the length of the string.	<pre>str = "Hello" print(len(str))</pre> OUTPUT 5
<code>ljust(width[, fillchar])</code>	Returns a string left-justified to a total of width columns. Columns without characters are padded with the character specified in the fillchar argument.	<pre>str = "Hello" print(str.ljust(10, '*'))</pre> OUTPUT Hello*****
<code>rjust(width[, fillchar])</code>	Returns a string right-justified to a total of width columns. Columns without characters are padded with the character specified in the fillchar argument.	<pre>str = "Hello" print(str.rjust(10, '*'))</pre> OUTPUT *****Hello

TABLE 9.2 (Contd.)

Function	Usage	Example
<code>zfill (width)</code>	Returns string left padded with zeros to a total of width characters. It is used with numbers and also retains its sign (+ or -).	<pre>str = "1234" print(str.zfill(10))</pre> OUTPUT <pre>0000001234</pre>
<code>lower()</code>	Converts all characters in the string into lowercase.	<pre>str = "Hello" print(str.lower())</pre> OUTPUT <pre>Hello</pre>
<code>upper()</code>	Converts all characters in the string into uppercase.	<pre>str = "Hello" print(str.upper())</pre> OUTPUT <pre>HELLO</pre>
<code>lstrip()</code>	Removes all leading whitespace in string.	<pre>str = "Hello" print(str.lstrip())</pre> OUTPUT <pre>Hello</pre>
<code>rstrip()</code>	Removes all trailing whitespace in string.	<pre>str = " Hello "</pre> <pre>print(str.rstrip())</pre> OUTPUT <pre>Hello</pre>
<code>strip()</code>	Removes all leading and trailing whitespace in string.	<pre>str = "Hello "</pre> <pre>print(str.strip())</pre> OUTPUT <pre>Hello</pre>
<code>max(str)</code>	Returns the highest alphabetical character (having highest ASCII value) from the string str.	<pre>str = "hello friendz" print(max(str))</pre> OUTPUT <pre>Z</pre>
<code>min(str)</code>	Returns the lowest alphabetical character (lowest ASCII value) from the string str.	<pre>str = "hello friendz" print(min(str))</pre> OUTPUT <pre>D</pre>
<code>replace(old, new [, max])</code>	Replaces all or max (if given) occurrences of old in string with new.	<pre>str = "hello hello hello" print(str.replace("he", "FO"))</pre> OUTPUT <pre>F0lloF0lloF0llo</pre>

Programming Tip: The empty parentheses means that this method takes no argument.

Function	Usage	Example
title()	Returns string in title case.	<pre>str = "The world is beautiful" print(str.title())</pre> OUTPUT The World Is Beautiful
swapcase()	Toggles the case of every character (uppercase character becomes lowercase and vice versa).	<pre>str = "The World Is Beautiful" print(str.swapcase())</pre> OUTPUT the world is beautiful
split(delim)	Returns a list of substrings separated by the specified delimiter. If no delimiter is specified then by default it splits strings on all whitespace characters.	<pre>str = "abc,def, ghi,jkl" print(str.split(','))</pre> OUTPUT ['abc', 'def', ' ghi', 'jkl']
join(list)	It is just the opposite of split. The function joins a list of strings using the delimiter with which the function is invoked.	<pre>print('-'.join(['abc', 'def', 'ghi', 'jkl']))</pre> OUTPUT abc-def-ghi-jkl
isidentifier()	Returns True if the string is a valid identifier.	<pre>str = "Hello" print(str.isidentifier())</pre> OUTPUT True
enumerate(str)	Returns an enumerate object that lists the index and value of all the characters in the string as pairs.	<pre>str = "Hello wOrld" print(list(enumerate(str)))</pre> OUTPUT [(0, 'H'), (1, 'e'), (2, 'l'), (3, 'l'), (4, 'o'), (5, ' '), (6, 'w'), (7, 'O'), (8, 'r'), (9, 'l'), (10, 'd')]

to see contents of string module

In [67]:

```
import string
dir(string)
```

```
'Template',
'_ChainMap',
'__all__',
'__builtins__',
'__cached__',
'__doc__',
'__file__',
'__loader__',
'__name__',
'__package__',
'__spec__',
'_re',
'_sentinel_dict',
'_string',
'ascii_letters',
'ascii_lowercase',
'ascii_uppercase',
'capwords',
'digits',
'hexdigits',
'octdigits',
'printable',
'punctuation',
'whitespace']
```

```
In [68]: str="hello"
         print(dir(str))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattr__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

String capitalize() method

This method provides us a copy of the string with the first letter capitalized and the rest lowercase as the output.

Syntax:

```
str.capitalize()
```

Parameters

NA

Return value

string

```
In [42]: s="a branch is an instruction in a computer program that can cause a computer to  
s.capitalize()
```

```
Out[42]: 'A branch is an instruction in a computer program that can cause a computer to  
begin executing a different instruction'
```

String title() method

The title() method returns a copy of the string in which the starting characters of all the words are capitalized.

Syntax:

str.title()

Parameters

NA

Return value

string

```
In [43]: s="A branch is an instruction in a computer program that can cause a computer to  
s.title()
```

```
Out[43]: 'A Branch Is An Instruction In A Computer Program That Can Cause A Computer To  
Begin Executing A Different Instruction'
```

String lower() method

The lower() method returns a copy of the string if all case-based characters have been lowercased.

Syntax:

str.lower()

Parameters

NA

Return value

string

```
In [44]: s="A branch is an instruction in a computer program that can cause a computer to  
s.lower()
```

Out[44]: 'a branch is an instruction in a computer program that can cause a computer to begin executing a different instruction'

String upper() method

The upper() method returns a copy of the string in which all case-based characters have been uppercased.

Syntax:

str.upper()

Parameters

NA

Return type

Returns the string with all characters converted to uppercase

```
In [45]: s="A branch is an instruction in a computer program that can cause a computer to  
s.upper()
```

```
Out[45]: 'A BRANCH IS AN INSTRUCTION IN A COMPUTER PROGRAM THAT CAN CAUSE A COMPUTER TO  
BEGIN EXECUTING A DIFFERENT INSTRUCTION'
```

String swapcase() method

Swapping the case of the characters in a string is accomplished by the swapcase() method.

Syntax:

str.swapcase()

Parameters

NA

Return type

If all the case-based characters are swapped, then this method returns a copy of the string

```
In [47]: s="A branch IS an instruction in a computer program that can cause a computer to  
s.swapcase()
```

```
Out[47]: 'a BRANCH is AN INSTRUCTION IN A COMPUTER PROGRAM THAT CAN CAUSE A COMPUTER TO  
BEGIN EXECUTING A DIFFERENT INSTRUCTION'
```

String count() method

The count() method gives the number of occurrences of the substring sub within the range [start, end] as the output. Arguments start and end are optional, and they are interpreted the same as in slice notation.

Syntax: str.count(sub, start= 0,end=len(string))

Parameters

sub - This is the substring needed for searching

start - Search always starts from this index, and the first character usually starts from the index, so search starts from the index by default.

end - Search ends at this index; the first character always starts from index, and search ends at the last index in the default case

Return value Centered in a string with length width

In [48]: `s="A branch is an instruction in a computer program that can cause a computer to
s.count("i")`

Out[48]: 9

In [49]: `s="A branch is an instruction in a computer program that can cause a computer to
s.count("i",15,50)`

Out[49]: 3

String find() method

The find() method determines whether the given string str occurs in the string or in a substring of the string if the starting index beg and ending index end are given.

Syntax: str.find(str, beg=0 end=len(string))

Parameters

str - It is the string needed to be found

beg - It is the starting index of the string; by default, it is zero

end - It indicates the ending index; by default, it is equal to the length of the string.

Return type

Index if found and -1 otherwise

In [50]: `s="A branch is an instruction in a computer program that can cause a computer to
s.find("in")`

Out[50]: 15

```
In [51]: s="A branch is an instruction in a computer program that can cause a computer to  
s.find("in",16,len(s))
```

```
Out[51]: 27
```

```
In [52]: s="A branch is an instruction in a computer program that can cause a computer to  
s.find("in",110,len(s))
```

```
Out[52]: -1
```

String rfind() method

The rfind() method returns the last index in which the str substring is found or -1 if no such index exists; optionally, the search is restricted to String

Syntax: str.rfind(str, beg=0 end=len(string))

Parameters

str – It specifies the string to be searched

beg - It is the starting index; by default, it is 0

end - By default, the ending index is equal to the length of the string

Return type

Index if found and -1 otherwise

```
In [53]: s="A branch is an instruction in a computer program that can cause a computer to  
s.rfind("in")
```

```
Out[53]: 106
```

```
In [54]: s="A branch is an instruction in a computer program that can cause a computer to  
s.rfind("in",5,100)
```

```
Out[54]: 90
```

```
In [55]: s="A branch is an instruction in a computer program that can cause a computer to  
s.rfind("in",0,7)
```

```
Out[55]: -1
```

String index() method

The index() method determines whether the string str occurs in string or in a substring of the string, given the starting index beg and ending index end. This method is similar to but it raises an exception if sub cannot be found.

Syntax: str.index(str, beg=0 end=len(string))

Parameters

Str: It specifies the string to be searched It is the starting index; by default, it is 0 The ending index, by default, is equal to the string length

Return value Index if and raises an exception if str is not found

```
In [56]: s="A branch is an instruction in a computer program that can cause a computer to
s.index("in")
```

```
Out[56]: 15
```

```
In [57]: s="A branch is an instruction in a computer program that can cause a computer to
s.index("in",16,len(s))
```

```
Out[57]: 27
```

```
In [58]: s="A branch is an instruction in a computer program that can cause a computer to
s.index("in",110,len(s))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[58], line 2
      1 s="A branch is an instruction in a computer program that can cause a comp
uter to begin executing a different instruction"
----> 2 s.index("in",110,len(s))

ValueError: substring not found
```

String rindex() method

When rindex() is called, it returns the last index where the str substring has been found or raises an exception if no index exists; optionally the search is restricted to

Syntax: str.rindex(str, beg=0 end=len(string))

Parameters

str - It specifies the string to be searched

beg – The starting index; by default, it is 0

len – The ending index; by default, it is equal to the length of the string

return index if or not

```
In [59]: s="A branch is an instruction in a computer program that can cause a computer to
s.rindex("in")
```

```
Out[59]: 106
```

```
In [60]: s="A branch is an instruction in a computer program that can cause a computer to
s.rindex("in",5,100)
```

Out[60]: 90

In [61]: `s="A branch is an instruction in a computer program that can cause a computer to
s.rindex("in",0,7)`

```
-----
ValueError                                Traceback (most recent call last)
Cell In[61], line 2
      1 s="A branch is an instruction in a computer program that can cause a comp
uter to begin executing a different instruction"
----> 2 s.rindex("in",0,7)

ValueError: substring not found
```

String startswith() method

The startswith() method checks whether the string starts with str, optionally restricting the matching with the given indices start and end.

Syntax: str.startswith(str, beg=0,end=len(string))

Parameters

str - String to be checked

beg - This is the optional parameter to set start index to matching boundaries

end - This is the optional parameter to set end index of the matching boundary

Return type This method returns true if found matching with the string; otherwise, it returns false.

In [62]: `'my name is nitish'.startswith('my')`

Out[62]: True

In [63]: `'my name is nitish'.startswith('lmy')`

Out[63]: False

In [64]: `'my name is nitish'.startswith('myn')`

Out[64]: False

In [65]: `'my name is nitish'.startswith('my n')`

Out[65]: True

String endswith() method

The boolean value true is returned when the string ends with the given suffix; otherwise, it returns false. Also, it restricts the matching with the given indices start and end, which are optional.

Syntax: `str.endswith(suffix[, start[, end]])`

Parameters

suffix – It can be a string or a tuple of suffixes to look for

start – It represents the start of the slice

end – It indicates the end of the slice

```
In [66]: 'my name is nitish'.endswith('sh')
```

```
Out[66]: True
```

```
In [67]: 'my name is nitish'.endswith('sho')
```

```
Out[67]: False
```

String `isalnum()` method

The `isalnum()` method checks whether the string contains alphanumeric characters.

Syntax: `str.isalnum()`

Parameters NA Return type This method returns true if all characters in the string are alphanumeric and there is at least one character; otherwise, it returns false.

```
In [68]: "vishalnot".isalnum()
```

```
Out[68]: True
```

```
In [69]: "12345698".isalnum()
```

```
Out[69]: True
```

```
In [70]: "vishnot123".isalnum()
```

```
Out[70]: True
```

```
In [75]: "vish not 123".isalnum()
```

```
Out[75]: False
```

String `isalpha()` method

The `isalpha()` method checks whether the string consists of alphabetic characters only.

Syntax: `str.isalpha()`

Parameters NA

Return type The method returns true if there is at least one character in the string and all characters are alphabetic; otherwise, it returns false.

```
In [72]: "vishalnot".isalpha()
```

```
Out[72]: True
```

```
In [73]: "12345698".isalpha()
```

```
Out[73]: False
```

```
In [74]: "vishnot123".isalpha()
```

```
Out[74]: False
```

String islower() method

You can check whether all case-based characters (letters) in a string are lowercase by calling the islower() method.

Syntax: str.islower()

Parameters NA

Return type

The method returns true if all cased characters in the string are lowercase

```
In [76]: "vishal".islower()
```

```
Out[76]: True
```

```
In [77]: "Vishal".islower()
```

```
Out[77]: False
```

```
In [78]: "vish123".islower()
```

```
Out[78]: True
```

String isnumeric() method and isdigit()

The isnumeric() method checks whether a string contains only numeric characters. It is only available on unicode objects.

Syntax: str.isnumeric()

Parameters NA

Return type

This method returns true if all characters are numeric; otherwise, it returns false.

```
In [79]: "123456".isnumeric()
```


Out[79]: True

```
In [80]: "vish123".isnumeric()
```

Out[80]: False

```
In [81]: "123456".isdigit()
```

Out[81]: True

```
In [82]: "vish123".isdigit()
```

Out[82]: False

String isspace() method

The isspace() method checks whether the string consists of whitespace.

Syntax: str.isspace()

Parameters NA

Return type

This method returns true if there is at least one character in the string and there are only whitespace characters; otherwise, it returns false.

```
In [83]: " ".isspace()
```

Out[83]: True

```
In [84]: " vish not".isspace()
```

Out[84]: False

String istitle() method

The istitle() method checks whether all case-based characters in a string following non-case-based letters are uppercase and whether all other case-based characters are lowercase.

Syntax: str.istitle()

Parameters NA

Return type

As an example, uppercase characters can only follow uncased characters, and lowercase characters can only follow cased characters. So, this method returns true if the string is a title cased string and there is at least one character; otherwise, it returns false.

```
In [85]: "Vishal Acharya".istitle()
```

Out[85]: True

In [86]: `"Vishal acharya".istitle()`

Out[86]: False

String isupper() method

The isupper() method determines whether all the case-based characters (letters) in the string are uppercase.

Syntax: str.isupper()

Parameters NA

Return type The method returns true only if all characters in the string are uppercase

In [87]: `"VISHAL".isupper()`

Out[87]: True

In [88]: `"ViSHAL".isupper()`

Out[88]: False

string isidentifier() method

In [89]: `"1nb".isidentifier()`

Out[89]: False

In [90]: `"vish1".isidentifier()`

Out[90]: True

In [91]: `"True".isidentifier()`

Out[91]: True

In [92]: `"sum".isidentifier()`

Out[92]: True

String join() method

The join() method returns a string containing all the string elements of a sequence separated by an str separator.

Syntax: str.join(sequence)

Parameters sequence - In this case, the elements will be joined in a sequence

Return type

The method returns a string that is the concatenation of the strings in the sequence seq. The string providing this method acts as a separator between elements.

```
In [93]: " ".join(['hi', 'my', 'name', 'is', 'vishal'])
```

```
Out[93]: 'hi my name is vishal'
```

```
In [94]: "vishal".join("acharya")
```

```
Out[94]: 'avishalcvishalh Vishalavishalrvishalyvishala'
```

```
In [95]: str1="vishal"
str2="acharya"
str1.join(str2)
```

```
Out[95]: 'avishalcvishalh Vishalavishalrvishalyvishala'
```

```
In [96]: "<".join("123")
```

```
Out[96]: '1<2<3'
```

String split() method

The split() function splits a string into words using str as a separator (breaks on whitespace if not specified), there is an option limiting the number of splits to num.

Syntax: str.split(str="", num=string.count(str))

Parameters str - It is any delimiter; by default, this is space This is the number of lines to be made

Return type This method returns a list of lines

```
In [98]: s="A branch is an instruction in a computer program that can cause a computer to
print(s.split())
```

```
['A', 'branch', 'is', 'an', 'instruction', 'in', 'a', 'computer', 'program', 'tha
t', 'can', 'cause', 'a', 'computer', 'to', 'begin', 'executing', 'a', 'differen
t', 'instruction']
```

```
In [100... s="A branch is an instruction in a computer program that can cause a computer to
print(s.split("in"))
```

```
['A branch is an ', 'struction ', ' a computer program that can cause a computer
to beg', ' execut', 'g a different ', 'struction']
```

```
In [101... s="A branch, is an instruction in a computer, program that can cause, a computer
print(s.split(","))
```

```
['A branch', ' is an instruction in a computer', ' program that can cause', ' a c
omputer to begin executing', ' a different instruction']
```

Syntax:

str.replace(old, new[, max])

Parameters

old – This is the old substring to be replaced

new - This is a new substring that replaces the old substrings

max - When the optional argument max is given, only the first count occurrences are replaced

Return type This method returns a copy of the string with all instances of substring old replaced by the new one. By specifying the optional argument max, only the first count occurrences are replaced.

```
In [102... s="A branch is an instruction in a computer program that can cause a computer to
s.replace("in","of")
```

```
Out[102... 'A branch is an ofstruction of a computer program that can cause a computer to
begin executofg a different ofstruction'
```

```
In [103... s="A branch is an instruction in a computer program that can cause a computer to
s.replace("in","of",2)
```

```
Out[103... 'A branch is an ofstruction of a computer program that can cause a computer to
begin executing a different instruction'
```

String lstrip() method

The lstrip() method returns a copy of the string that has been stripped of all characters at the beginning (default whitespace characters).

Syntax: str.lstrip([chars])

Parameters chars - You can specify which characters need to be trimmed.s

```
In [104... "abcvishal acharya".lstrip("abc")
```

```
Out[104... 'vishal acharya'
```

```
In [105... "abcvishal acharya".rstrip("a")
```

```
Out[105... 'abcvishal achary'
```

```
In [106... " abcvishal".lstrip()
```

```
Out[106... 'abcvishal'
```

```
In [107... String rstrip() method
```

Whitespace characters are stripped **from** the string using rstrip() (default).

Syntax:

str.rstrip([chars])

Parameters

chars - You can give what chars have to be trimmed

Cell In[107], line 2

^

SyntaxError: invalid non-printable character U+200B

In [108... `"abcvishal acharya".rstrip("ya")`

Out[108... `'abcvishal achar'`

In [109... `"abcvishal acharya ".rstrip(" ")`

Out[109... `'abcvishal acharya'`

In [110... `"abcvishal acharya ".rstrip("a")`

Out[110... `'abcvishal acharya '`

String strip() method

The Strip() method returns a copy of a string in which all whitespace characters have been stripped (default).

Syntax: `str.strip([chars])`

Parameters chars - Characters will be trimmed at the start or the end with the selected characters.

In [111... `"abcvishal acharya".strip("a")`

Out[111... `'bcvishal achary'`

In [112... `" abcvishal acharya ".strip(" ")`

Out[112... `'abcvishal acharya'`

In [113... `" abcvishal acharya ".strip("a")`

Out[113... `' abcvishal acharya '`

String maketrans() method

Using each character in the intabstring is mapped into the character in the outtabstring at the same position. Translate() then takes this table as input.

Syntax: `str.maketrans(intab, outtab);`

Parameters intab - The actual characters are in this string outtab - There are corresponding mapping characters in this string

Return type The method returns a translate table to be used with the `translate()` function

```
In [114... intab="aeiou"
outtab = "12345"
my = "this is string translate function...!!"
trantab =my.maketrans(intab, outtab)
print (my.translate(trantab))
```

th3s 3s str3ng tr1nsl1t2 f5nct34n...!!

String translate() method

A copy of the string is returned by the translate() method, in which the characters have been translated using table (created by maketrans() in the string module), optionally removing any characters found in the string

Syntax: str.translate(table[, deletechars]);

Parameters table can use the maketrans() function to create translate table deletechars - Several characters in the source string need to be removed

Return type This method returns a copy of the translated string

```
In [116... import string

a_string = '!hi. wh?at is the weat[h]er lik?e.'
new_string = a_string.translate(str.maketrans('','',string.punctuation))

print(new_string)
```

hi what is the weather like

```
In [117... # Find the Length of a given string without using the Len() function
s = input('enter the string')
counter = 0
for i in s:
    counter += 1
print('length of string is',counter)
```

length of string is 14

```
In [118... # Extract username from a given email.
# Eg if the email is vishal76@gmail.com
# then the username should be vishal76@gmail.com
s = input('enter the email')
pos = s.index('@')
print(s[0:pos])
```

vishal76

```
In [120... # Count the frequency of a particular character in a provided string.
# Eg 'hello how are you' is the string, the frequency of h in this string is 2.
s = input('enter the email')
term = input('what would like to search for')
counter = 0
for i in s:
    if i == term:
        counter += 1
print('frequency',counter)
```

frequency 3

```
In [121... # Write a program which can remove a particular character from a string.
s = input('enter the string: ')
term = input('what would like to remove: ')
result = ''
for i in s:
    if i != term:
        result = result + i
print(result)
```

vishl chry

```
In [122... # Write a program that can check whether a given string is palindrome or not.
# abba
# malayalam
s = input('enter the string')
flag = True
for i in range(0, len(s)//2):
    if s[i] != s[len(s) - i - 1]:
        flag = False
        print('Not a Palindrome')
        break

if flag:
    print('Palindrome')
```

Palindrome

```
In [123... # Write a program to count the number of words in a string without split()

s = input('enter the string')
L = []
temp = ''
for i in s:
    if i != ' ':
        temp = temp + i
    else:
        L.append(temp)
        temp = ''
L.append(temp)
print(L)
```

['how', 'are', 'you']

```
In [124... # Write a program to count the number of words in a string without split()

s = input('enter the string')
L = []
temp = ''
for i in s:
    if i != ' ':
        temp = temp + i
    else:
        L.append(temp)
        temp = ''
L.append(temp)
print(L)
print(len(L))
```

```
['hello', 'how', 'are', 'you']
4
```

```
In [125]: # Write a python program to convert a string to title case without using the tit
s = input('enter the string')
L = []
for i in s.split():
    L.append(i[0].upper() + i[1:].lower())
print(" ".join(L))
```

Hello Vishal B1 B3 B7

```
In [17]: # Write a program that can convert an integer to string.
```

```
number = int(input('enter the number'))

digits = '0123456789'
result = ''
while number != 0:
    result = digits[number % 10] + result
    number = number//10

print(result)
print(type(result))
```

enter the number12345
12345
<class 'str'>

```
In [18]: #Write a program to create a string made of first,middle and last character
```

```
s=input("enter string: ")
new_string=""
new_string=s[0]+s[len(s)//2]+s[-1]
print(new_string)
```

enter string: vishalacharya
vaa

```
In [19]: #Write a program to find all occuences of a sub string in a given string by igno
```

```
s=input("enter string: ")
a=input("sub string: ")
print(s.upper().count((a.upper())))
```

enter string: lj university
sub string: I
2

```
In [20]: #Write a program to calculate the sum and average of the digits present in a str
```

```
s=(input("enter string: "))
sum=0
for i in s:
    if i.isdigit()==True:
        sum+=int(i)
print(sum)
```

enter string: vish123 lki25 457
29

```
In [21]: k=(input("enter string: "))
```

```
sum=0
s=k.split(" ")
for i in s:
```



```

    if i.isdigit()==True:
        sum+=int(i)
print(sum)

```

enter string: vish123 lki25 457
457

In [23]: *#Write a Python program to print even length words in a string.*

```

s=input("enter string: ")
n=s.split(" ")
for i in n:
    if len(i)%2==0:
        print(i,end=" ")

```

enter string: computer fsd python hgy kju lkio lkio
computer python lkio lkio

In [24]: *#Write a Python program to Uppercase Half String from the given string.*

```

s=input("enter string: ")
new_string1=s[0:len(s)//2:1].upper()
new_string2=s[len(s)//2:len(s)+1:1]
print(new_string1+new_string2)

```

enter string: computer fsd python hgy kju lkio lkio
COMPUTER FSD PYTHOn hgy kju lkio lkio

In [25]: *#Write a Python program to capitalize the first and last character of each word*

```

s=input("enter a string")
s = s.title()
result = ""
for word in s.split():
    result += word[:1] + word[-1].upper() + " "
print(result)

```

enter a stringlkibngvv ki mnjh
LkibngvV KI MnjH

In [26]: *#Write a program to Create a string made of the middle three characters*

```

s=input("enter string: ")
new_string=s[(len(s)//2)-1:(len(s)//2)+2]
print(new_string)

```

enter string: jhuykjj jhuykjj jhuykjj jhuykjj
j j

In [27]: *"""Write a program to check if two strings are balanced. For example, strings s1 and s2 are balanced if all the characters in the s1 are present in s2 The character's position doesn't matter."""*

```

s1=input("enter string: ")
s2= input("enter string: ")
flag=0
for i in s1:
    if i in s2:
        continue
    else:
        flag=1
if(flag ==1):
    print("not balanced")
else:
    print("balanced")

```

```
enter string: lkju
enter string: lk
not balanced
```

In [28]: `"""Write a program to check if two strings are balanced. For example, strings s1 and s2 are balanced if all the characters in the s1 are present in s2. The character's position doesn't matter."""`

```
s1=input("enter string: ")
s2= input("enter string: ")
flag=0
for i in s1:
    if i in s2:
        continue
    else:
        flag=1
if(flag ==1):
    print("not balanced")
else:
    print("balanced")
```

```
enter string: lk
enter string: lkju
balanced
```

In [29]: `#Write a program to Split a string on hyphens`

```
s1=input("enter string: ")
s2=s1.split("-")
print(s2)
```

```
enter string: hy]oi-uy-iu
['hy]oi', 'uy', 'iu']
```

In [30]: `#Replace each special symbol with # in the following string`

```
import string
s=input("enter string: ")
for i in string.punctuation:
    s=s.replace(i,"#")
print(s)
```

```
enter string: kjk* &^
kjk# ##
```

In [31]: `import string`

```
test_str = 'vid, is best: for ! gjhks ;'

test_str = test_str.translate(str.maketrans('', '', string.punctuation))
print(test_str)
```

```
vid is best for  gjhks
```

In [32]: `"""Write program that takes user's name and pan card number as input validate information using is x function and print the detail"""`

```
def pan():
    pan_name=input("enter name")
    if pan_name.isalpha()==False:
        print("sorry, wrong name")

    pan_number= input("enter pan number: ")
    if pan_number.isalnum()==False:
        print("sorry,wrong number")
```

```
print("correct")

pan()
```

enter namejhjj
enter pan number: 1454
correct

In [33]: *#Wpp program that encrypts a message by adding a key value to every character.*

```
s="abcdefz "
index=0
while index<len(s):
    l=s[index]
    print(chr(ord(l)+3),end="")
    index+=1
```

defghi}#

In [34]: *#Wpp a program to generate abecedarian series*

```
s="ABCDEFGH"
s1="ate"
for i in s:

    print(i+s1)
```

Aate
Bate
Cate
Date
Eate
Fate
Gate
Hate

In [35]: *#Wpp that accept a string from user and redisplay the same string after removing*

```
s=input("enter string: ")
new=""
for i in s:
    if i in "aeiouAEIOU":
        continue
    else:
        new=new+i
print(new)
```

enter string: ljiet ljuniversity
ljt ljnvrsty

In [36]: *""Wpp that find whether a given character is present in a string or not. In case it is present it prints the index at which it is present do not use built in find function to search the character""*

```
s=input("enter string: ")
a=input("enter character:")
index=0
for i in s:
    if a==i:
        print(index,"of s",a)
        index+=1
```

enter string: jhujhu
enter character:j
0 of s j
3 of s j